strncmp()

Our last function to talk about here is not actually a function of the Serial libraries, but is rather a part of the standard C library on which Arduino is built. The function `strncmp()` is used to compare two text strings and is an example of **string manipulation**. We use this function in our example code to compare the text strings from two different tags to see if they match—kind of like a lock and key. While the Arduino has a `String` class available with much more advanced capabilities, this function works well enough for our purposes. It has the following syntax:

```
strncmp(string1, string2, numberOfCharacters)
```

This function will compare `string1` to `string2` up to the maximum number of characters specified. If the two strings are identical, this function will return the value 0. Take the following hypothetical example:

```
if (strncmp("goodnight", "room", 4) == 0) Serial.print("goodnight moon");
```

In this example, the `strncmp()` function would compare the string `"goodnight"` to the string `"moon"` up to a maximum four characters. If these two strings were the same, then the expression would evaluate as `true` and the following call to `Serial.print` would be executed. Each of these strings needs to be specified as either the string itself or a pointer to the string. In our example code, it is as follows:

```
if (strncmp(tag01, tagString, 12) == 0) return " Tag 1";
```

Here we are using the names of the arrays `tag01[]` and `tagString[]` as pointers to the first byte in the array and are then comparing the full 12 digits of each array. If these arrays match, then we can perform an action such as printing the tag's name or some other activity. If they do not match, we can continue to compare the tag that we just read with other known tags to see if there is a match.

While not generally discussed through the official Arduino channels, string manipulation and other functions of the standard C library can be quite useful in the right circumstances. There are many more string functions as well that can be found in a good book on C programming or with a little digging in the great interweb.

From here though, we are going to look at a little different kind of serial communication called I2C that is designed to talk with other kinds of hardware. While many different devices exist, we will start with an easy-to-use, real-time clock module that keeps an accurate time and date with a battery backup for standby operation. With the right commands, it will fairly accurately tell us "when" we are.

Project 11: Serial Time Clock

The Arduino has a fairly precise internal clock, but it's not the most well-suited device to keep track of the date and time. This can be a fairly useful ability to have if we want to keep track of the time the cat comes in to the house or we want to make another digital clock. To get a reasonably accurate date and time with a little backup, we need to use a real-time clock or RTC module. For this project, we will use the venerable DS1307 RTC available on a breakout board from SparkFun Electronics and Adafruit Industries.

The DS1307 is a relatively simple device that uses the Inter-Integrated Circuit communication protocol, or I2C, or sometimes Two Wire—although these two terms are not always interchangeable—to send a 7-bit binary number for the current date and time. I2C is an interesting serial protocol that uses two wires, SDA for data and SCL for clock, to control up to 112 slave devices such as sensors, clocks, FM radios, and smart LEDs, from one master device like our microcontroller. In addition to the data and clock pins, each I2C device will also need a positive and ground connection to a power supply rated for the device. The I2C pins on the Arduino interface board, although not explicitly marked as such, are pins